WEEK 2 WRITEUP

I Made Ananta Wijaya
NIM: 260530911102
Divisi: Cybersecurity

DAFTAR ISI

vault-door-training....3
Transformation....4
Flag Hunters....6
Gatekeeper....9
Hidden Cipher 2....12
Hashgate....14
Unminify....15

vault-door-training

Kategori: Reverse Engineering

Deskripsi: Your mission is to enter Dr. Evil's laboratory and retrieve the blueprints for his Doomsday Project. The laboratory is protected by a series of locked vault doors. Each door is controlled by a computer and requires a password to open. Unfortunately, our undercover agents have not been able to obtain the secret passwords for the vault doors, but one of our junior agents obtained the source code for each vault's computer! You will need to read the source code for each level to figure out what the password is for that vault door. As a warmup, we have created a replica vault in our training facility.

Tools yang digunakan: VS Code

Challenge ini memberikan kita sebuah source code dalam bahasa Java. Berikut adalah tampilannya di VS Code.



```java
import java.util.*;

class VaultDoorTraining {
    Run | Debug
    public static void main(String args[]) {
        VaultDoorTraining vaultDoor = new VaultDoorTraining();
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter vault password: ");
        String userInput = scanner.next();
    String input = userInput.substring("academy{".length(),userInput.length()-1);
    if (vaultDoor.checkPassword(input)) {
        System.out.println("Access granted.");
    } else {
        System.out.println("Access denied!");
    }
   }

    // The password is below. Is it safe to put the password in the source code?
    // What if somebody stole our source code? Then they would know what our
    // password is. Hmm... I will think of some ways to improve the security
    // on the other doors.
    //
    // -Minion □#9567
    public boolean checkPassword(String password) {
        return password.equals("w4rm1ng_Up_w1tH_jAv4_69bd4dc7adf");
    }
}
```

Class VaultDoorTraining mengambil input password untuk vault lalu dicek menggunakan fungsi checkPassword(). Fungsi checkPassword() mengecek apakah password yang diberikan sama dengan "`w4rm1ng_Up_w1tH_jAv4_69bd4dc7adf`". Jadi apa itu password yang harus diinputkan user? Perhatikan kode untuk mengambil input user.

```
userInput.substring("academy{".length(),userInput.length()-1);
```

substring() digunakan untuk mengambil sebagian karakter dari index tertentu sampai index lain. Dalam kode tersebut, substring mengambil input user dari index 8 (panjang dari “academy{”) hingga sebelum karakter terakhir (karakter terakhir dibuang). Jadi, input user dipotong dulu 8 karakter awalnya dan 1 karakter akhirnya, baru dicek menggunakan fungsi checkPassword(). Jadi yang harus diinputkan user adalah:
academy{w4rm1ng_Up_w1tH_jAv4_69bd4dc7adf}

Bagian “academy{“ dan “}” akan dibuang dulu oleh fungsi substring dan w4rm1ng_Up_w1tH_jAv4_69bd4dc7adf akan dicek oleh fungsi checkPassword().

Flag: academy{w4rm1ng_Up_w1tH_jAv4_69bd4dc7adf}

Transformation

Kategori: Reverse Engineering

Deskripsi: I wonder what this really is...

Tools yang digunakan: VS Code

Challenge ini memberikan kita sebuah file bernama enc dan kode berikut:

```
''.join([chr((ord(flag[i]) << 8) + ord(flag[i + 1])) for i in range(0, len(flag), 2)])
```

Kode tersebut memiliki beberapa fungsi terkait dengan string

- join() adalah fungsi untuk menggabungkan beberapa string menjadi satu
- chr() adalah fungsi untuk mengubah integer menjadi char ASCII
- ord() adalah fungsi untuk mengubah char ASCII menjadi integer
- Operator << digunakan untuk melakukan bit shift ke kiri sebanyak 8 bit, atau jika dilakukan ke integer, sama dengan melakukan perkalian 256 dengan integer tersebut
- List comprehension, yang digunakan untuk langsung mengisi sebuah list dengan elemen
- len() digunakan untuk menghitung banyaknya char dalam string

Yang belum dibahas adalah flag, yang bisa kita asumsikan sebagai variabel string input. Karena yang diambil adalah variabel flag, kita bisa asumsikan bahwa kode tersebut adalah sebuah encoder yang menerima flag dan memberikan sebuah string lain, yaitu file enc. Coba kita lihat isi dari file enc

```
慣慤敨禂ら形楴獟楮獠㹧摟潦弸強恋扡ベッ敲
```

File enc tersebut berisi sebuah text dalam bentuk Unicode, yang kita bisa asumsikan sebagai output dari kode diatas. Jadi, kita harus mencari inputnya untuk mendapatkan flag.

KIta coba pisahkan kode diatas supaya lebih mudah dimengerti

```
flag = "unknown_"
joinedList = []

for i in range(0, len(flag), 2):
   currentChar = flag[i]
   nextChar = flag[i+1]

   currentCharNum = ord(currentChar)
   nextCharNum = ord(nextChar)

   currentCharNumBitShift = currentCharNum << 8 # or times 256

   joinedList.append(chr(currentCharNumBitShift + nextCharNum))

encrypted = ''.join(joinedList)
print(encrypted)
```

Tiap for loop, alurnya seperti ini

1. Ubah char sekarang dan char setelahnya menjadi integer
2. Char setelahnya, yang sudah diubah menjadi integer, di-bit shift sebanyak 8 ke kiri
3. Char sekarang (integer) ditambah char setelahnya (integer)
4. Hasil pertambahan tersebut diubah kembali menjadi char dan di-append ke string hasil

Sebagai contoh, untuk input
`flag = "unknown_"`
output yang diberikan adalah:
`畮庖潮湟`

Tugas kita sekarang adalah membuat dekoder yang sesuai
Alur dekodernya adalah kebalikan dari encodernya

1. Ubah tiap char menjadi angka
2. Angka tersebut diubah kembali menjadi dua angka input/sumbernya
3. Char sumber setelahnya (angka) di-bit shift sebanyak 8 bit ke kanan
4. Ubah tiap angka menjadi char kembali

Namun, bagaimana cara mengubah satu angka kembali menjadi dua angka kembali?

Seperti kita ketahui, char pertama sudah mengalami bit shift 8 kali ke kiri, lalu ditambah dengan char kedua, kedua char tersebut berukuran 8 bit. Berikut adalah visualisasinya

```
97  =  01100001
99  =  01100011

97 << 8  = 01100001      00000000
99       = 00000000      01100011
```

```
---------------------------------- + (ditambah)
           01100001 (97) 01100011 (99)
```

Kita bisa lihat, karena angka pertama sudah di-bit shift atau dipindahkan ke kiri, angka hasilnya berukuran 16 bit. Angka pertama berada di 8 bit pertama dan angka kedua ada di 8 bit setelahnya.
Jadi, untuk mengambil angka pertama, kita ambil 8 bit pertama dari angka hasil dan untuk mengambil angka kedua, kita ambil 8 bit kedua dari angka hasil.

```
firstCharNum  = currentCharNum >> 8
secondCharNum = currentCharNum & 0xFF
```

Setelah itu, kita tinggal ubah firstCharNum dan secondCharNum menjadi char kembali menggunakan fungsi ord() dan menambahkannya ke string hasil

```
firstCharNum  = currentCharNum >> 8
secondCharNum = currentCharNum & 0xFF
```

Berikut adalah kode untuk decodernya

```
enc = "慣慤敫裯ら形楴獟楮獳㹲摟潦弸強您扡ゃ敲"
joinedList = []

for i in range(0, len(enc)):
   currentChar = enc[i]

   currentCharNum = ord(currentChar)

   firstCharNum  = currentCharNum >> 8
   secondCharNum = currentCharNum & 0xFF

   joinedList.append(chr(firstCharNum))
   joinedList.append(chr(secondCharNum))

flag = ''.join(joinedList)
print(flag)
```

Flag = `academy{16_bits_inss34d_of_8_790ba37e}`

Flag Hunters

Kategori: Reverse Engineering

Deskripsi: Lyrics jump from verses to the refrain kind of like a subroutine call. There's a hidden refrain this program doesn't print by default. Can you get it to print it? There might be something in it for you.

Tools yang digunakan: VS Code

Challenge ini memberikan kita sebuah source code program dalam bahasa Python. Program tersebut membacakan sebuah lirik lagu baris per baris mulai dari verse pertama. Ada sebuah

intro rahasia yang tidak dijalankan oleh kode yang berisi flag yang kita cari. Berikut adalah logika utama dari kode

```
MAX_LINES = 100

def reader(song, startLabel):
 lip = 0
 start = 0
 refrain = 0
 refrain_return = 0
 finished = False

 # Get list of lyric lines
 song_lines = song.splitlines()
  # Find startLabel, refrain and refrain return
 for i in range(0, len(song_lines)):
   if song_lines[i] == startLabel:
     start = i + 1
   elif song_lines[i] == '[REFRAIN]':
     refrain = i + 1
   elif song_lines[i] == 'RETURN':
     refrain_return = i

 # Print lyrics
 line_count = 0
 lip = start
 while not finished and line_count < MAX_LINES:
   line_count += 1
   for line in song_lines[lip].split(';'):
     if line == '' and song_lines[lip] != '':
       continue
     if line == 'REFRAIN':
       song_lines[refrain_return] = 'RETURN ' + str(lip + 1)
       lip = refrain
     elif re.match(r"CROWD.*", line):
       crowd = input('Crowd: ')
       song_lines[lip] = 'Crowd: ' + crowd
       lip += 1
     elif re.match(r"RETURN [0-9]+", line):
       lip = int(line.split()[1])
     elif line == 'END':
       finished = True
     else:
       print(line, flush=True)
       time.sleep(0.5)
       lip += 1

reader(song_flag_hunters, '[VERSE1]')
```

Intro rahasianya adalah

```
secret_intro = \
'''Pico warriors rising, puzzles laid bare,
Solving each challenge with precision and flair.
With unity and skill, flags we deliver,
The ether's ours to conquer, '''\
+ flag + '\n'
```

Tugas kita adalah mencari input khusus supaya secret input di awal input (sebelum VERSE 1) itu dijalankan.

Perhatikan kode berikut

```
  for line in song_lines[lip].split(';'):
    if line == '' and song_lines[lip] != '':
      continue
    if line == 'REFRAIN':
      song_lines[refrain_return] = 'RETURN ' + str(lip + 1)
      lip = refrain
    elif re.match(r"CROWD.*", line):
      crowd = input('Crowd: ')
      song_lines[lip] = 'Crowd: ' + crowd
      lip += 1
    elif re.match(r"RETURN [0-9]+", line):
      lip = int(line.split()[1])
    elif line == 'END':
      finished = True
    else:
      print(line, flush=True)
      time.sleep(0.5)
      lip += 1
```

Tiap baris akan dibagi menjadi beberapa bagian, dipisahkan oleh simbol ;

Jika baris tersebut bertuliskan REFRAIN, lirik baris yang ada di refrain_return akan diubah menjadi 'RETURN (lip + 1)', lip itu menyatakan nomor baris saat ini.

Jika baris saat ini berisi "CROWD", program akan mengambil input user dan menempatkannya di "Crowd: (input user)".

Jika baris saat ini berisi "RETURN [0-9]" ([0-9 maksudnya nomor 0-9, jadi RETURN 0, RETURN 1, etc), maka lip diubah menjadi menjadi angka setelah "RETURN"

Kita bisa anggap REFRAIN, CROWD, dan RETURN itu sebagai fungsi. Perhatikan baris berikut:

```
  for line in song_lines[lip].split(';'):
```

Ini artinya satu baris bisa menerima lebih dari satu fungsi.

Supaya program memanggil secret intro, maka program harus memanggil RETURN 0. Karena tidak ada validasi, kita bisa menginputkan fungsi kita sendiri saat CROWD dipanggil.

Kita tinggal inputkan “bebas;RETURN 0” saat program meminta input.

Flag: `academy{70637h3r_f0r3v3r_697ad4d5}`

Gatekeeper

Kategori: Reverse Engineering

Deskripsi: What’s behind the numeric gate? You only get access if you enter the right kind of number.

Tools yang digunakan: Binary Ninja

Challenge ini memberikan kita sebuah file program c bernama gatekeeper. Coba kita jalankan.

```
→ ./gatekeeper
Enter a numeric code (must be > 999 ): 9998
Access Denied.
```

Program meminta angka yang lebih dari 999, tapi walaupun angka yang diberikan sudah memenuhi, akses masih tidak diterima.
KIta dekompilasi program tersebut menggunakan Binary Ninja.

```
0040159a    void* fsbase
0040159a    int64_t rax = *(fsbase + 0x28)
004015b8    printf(format: "Enter a numeric code (must be > 999 ): ")
004015c7    fflush(fp: __TMC_END__)
004015e2    char nptr[0x28]
004015e2    __isoc99_scanf(format: "%31s", &nptr)
004015e7    int32_t var_40 = 0xffffffff
004015f5    int32_t rax_3 = strlen(&nptr)
0040160b    int32_t result
0040160b    int32_t var_40_1
0040160b
0040160b    if (is_valid_decimal(&nptr) == 0)
0040162c        if (is_valid_hex(&nptr) != 0)
00401644            var_40_1 = strtol(&nptr, endptr: nullptr, base: 0x10)
00401647            goto label_401666
00401647
00401653        puts(str: "Invalid input.")
00401658        result = 1
0040160b    else
00401619        var_40_1 = atoi(&nptr)
00401666    label_401666:
00401666
00401666        if (var_40_1 s<= 0x3e7)
00401672            puts(str: "Too small.")
00401666        else if (var_40_1 s> 0x270f)
0040168c            puts(str: "Too high.")
00401680        else if (rax_3 != 3)
004016af            puts(str: "Access Denied.")
00401697        else
0040169e            reveal_flag()
0040169e
004016b4        result = 0
004016bd    *(fsbase + 0x28)
004016bd
004016c6    if (rax == *(fsbase + 0x28))
004016ce        return result
004016ce
```

```
004015e2        char nptr[0x28]
004015e2        __isoc99_scanf(format: "%31s", &nptr)
```

Pertama, program mengambil input user dan menyimpannya di nptr

```
004015f5        int32_t rax_3 = strlen(&nptr)
0040160b        int32_t result
0040160b        int32_t var_40_1
```
Program membuat variabel rax_3 untuk menyimpan panjang dari input

```
0040160b        if (is_valid_decimal(&nptr) == 0)
0040162c            if (is_valid_hex(&nptr) != 0)
00401644                var_40_1 = strtol(&nptr, endptr: nullptr, base: 0x10)
00401647                goto label_401666
00401647
00401653            puts(str: "Invalid input.")
00401658            result = 1
0040160b        else
00401619            var_40_1 = atoi(&nptr)
00401666        label_401666:
00401666
00401666            if (var_40_1 s<= 0x3e7)
00401672                puts(str: "Too small.")
00401666            else if (var_40_1 s> 0x270f)
0040168c                puts(str: "Too high.")
00401680            else if (rax_3 != 3)
004016af                puts(str: "Access Denied.")
00401697            else
0040169e                reveal_flag()
```

Jika kita perhatikan, di kedua kasus tersebut (Jika input bukan desimal valid maupun valid), keduanya masuk ke label_401666 (logika utama). Ini berarti input yang kita masukkan bisa desimal dan hexadesimal.

Perhatikan berikut

```
00401666            if (var_40_1 s<= 0x3e7)
00401672                puts(str: "Too small.")
00401666            else if (var_40_1 s> 0x270f)
0040168c                puts(str: "Too high.")
00401680            else if (rax_3 != 3)
004016af                puts(str: "Access Denied.")
00401697            else
0040169e                reveal_flag()
```

Jika input lebih kecil dari 999 (0x3e7), akan diberi error “Too Small”
Jika input lebih besar dari 9999 (0x270f), akan diberi error “Too high”
Jika panjang dari input (rex_3) lebih dari 3 karakter, akan diberi error “Access Denied.”

Supaya reveal_flag() bisa berjalan, maka input harus:

- Lebih besar dari 999
- Lebih kecil dari 9999
- Memiliki kurang atau sama dengan 3 karakter

Tapi, kriteria itu tidak bisa dipenuhi, karena input terkecil yang lebih besar dari 999 adalah 1000, yang lebih besar dari 3 karakter.

Jadi, kita gunakan bentuk input yang satunya lagi, yaitu heksadesimal

00401644 var_40_1 = strtol(&nptr, endptr: nullptr, base: 0x10)
var_40_ 1 diisi oleh input user yang sudah dikonversi ke heksadesimal dengan base 16 (0x10)

Supaya reveal_flag() bisa berjalan, maka input harus:

- Lebih besar dari 999
- Lebih kecil dari 9999
- Memiliki kurang atau sama dengan 3 karakter dalam heksadesimal.

Salah satu bilangan heksadesimal yang memenuhi adalah 3E8.
Coba kita inputkan




```
Access granted:
}deaftc_oc_ip8e90ftc_oc_ipa_99ftc_oc_ip9_TGftc_oc_ip_xehftc_oc_ip_tig
ftc_oc_ipid_3ftc_oc_ip{ymeftc_oc_ipdacaftc_oc_ip
```

Sepertinya, output yang diberikan itu belum benar. Kita gunakan hint:
The program’s output isn’t straightforward; reversing the string and cleaning out extra text may help you recover the flag.
Kita coba balikkan textnya:
pi_co_ctfacadpi_co_ctfemy{pi_co_ctf3_dipi_co_ctfgit_pi_co_ctfhex_pi_co_ctfGT_9pi_co_ctf99_api_co_ctf09e8pi_co_ctfaed}

Kalau kita perhatikan, tulisan pi_co_ctf terus diulang-ulang di dalam teks, kalau kita hapus:
academy{3_digit_hex_GT_999_a09e8aed}

Flag: academy{3_digit_hex_GT_999_a09e8aed}

Hidden Cipher 2

Kategori: Reverse Engineering

Deskripsi: The flag is right in front of you... kind of. You just need to solve a basic math problem to see it. But to get the real flag, you'll have to understand how that math answer is used.

Tools yang digunakan: Binary Ninja

Challenge ini memberikan kita sebuah file program c bernama hiddencipher2. Berikut adalah tampilannya

```
What is 1 * 5? 5
Encoded flag values:
485, 495, 485, 500, 505, 545, 605, 615, 545, 260, 580, 520, 475, 490, 255
, 520, 245, 550, 500, 475, 495, 245, 560, 520, 255, 570, 475, 275, 485, 2
65, 250, 270, 285, 270, 260, 625
```

Program tersebut menanyakan hasil dari suatu operasi matematika lalu memberikan flag yang kita butuhkan dalam bentuk angka yang sudah di-encode. Supaya kita tau flag asli, kita harus tau bagaimana program melakukan proses encoding.
KIta dekompilasi program tersebut menggunakan Binary Ninja.
Perhatikan kode berikut:

```
004016c3    int32_t rax_3 = generate_math_question(&var_29, &var_28, &var_24)
…
0040171b    int32_t var_20
0040171b    int32_t result
0040171b
0040171b    if (__isoc99_scanf(format: "%d", &var_20) != 1)
00401727        puts(str: "Invalid input. Exiting.")
…

0040171b        else if (rax_3 == var_20)
0040175b            void* rax_10 = read_flag_file("flag.txt")
0040175b
00401769            if (rax_10 != 0)
0040177e                encode_flag(rax_10, rax_3)
```

`rax_3` adalah hasil dari operasi matematika yang ditanyakan dan `var_20` adalah input yang dimasukkan oleh user. Kita bisa melihat bahwa flag yang ada di file tersebut di encode terlebih dahulu oleh fungsi `encode_flag()` sebelum di-print. Berikut adalah fungsi encode_flag

```
004014eb    int64_t encode_flag(void* arg1, int32_t arg2)
00401508        puts(str: "Encoded flag values:")
```

```
0040150d        int32_t var_c = 0
0040150d
00401584        while (*(arg1 + sx.q(var_c)) != 0)
0040153e            printf(format: "%d", zx.q(sx.d(*(arg1 + sx.q(var_c))) * arg2))
0040153e
00401558            if (*(arg1 + sx.q(var_c) + 1) != 0)
00401569                printf(format: ", ")
00401569
0040156e            var_c += 1
0040156e
00401592        return putchar(c: 0xa)
```

arg1 adalah flag yang ada di file dan arg2 adalah jawaban dari perhitungan matematika

Perhatikan bagian

```
00401584        while (*(arg1 + sx.q(var_c)) != 0)
0040153e            printf(format: "%d", zx.q(sx.d(*(arg1 + sx.q(var_c))) * arg2))
…
0040156e            var_c += 1
```

var_c adalah indeks dari while loop tersebut. Tiap iterasi, program akan print hasil dari arg1[1] (char di flag diubah ke int) dkali arg2 (hasil perhitungan matematika)

Jadi rumusnya adalah -> char di masing-masing indeks diubah jadi int * hasil perhitungan matematika. Untuk mengubahnya kembali ke flag maka tiap char di output program harus dibagi hasil perhitungan. Berikut adalah decodernya

```python
values = "485, 495, 485, 500, 505, 545, 605, 615, 545, 260, 580, 520, 475,
490, 255, 520, 245, 550, 500, 475, 495, 245, 560, 520, 255, 570, 475, 275,
485, 265, 250, 270, 285, 270, 260, 625".split(",")
flag = ""
for v in values:
   flag += chr(int(v) // 5)

print(flag)
```

Jika dijalankan, outputnya memberikan flag yang kita butuhkan

Flag: academy{m4th_b3h1nd_c1ph3r_7a526964}

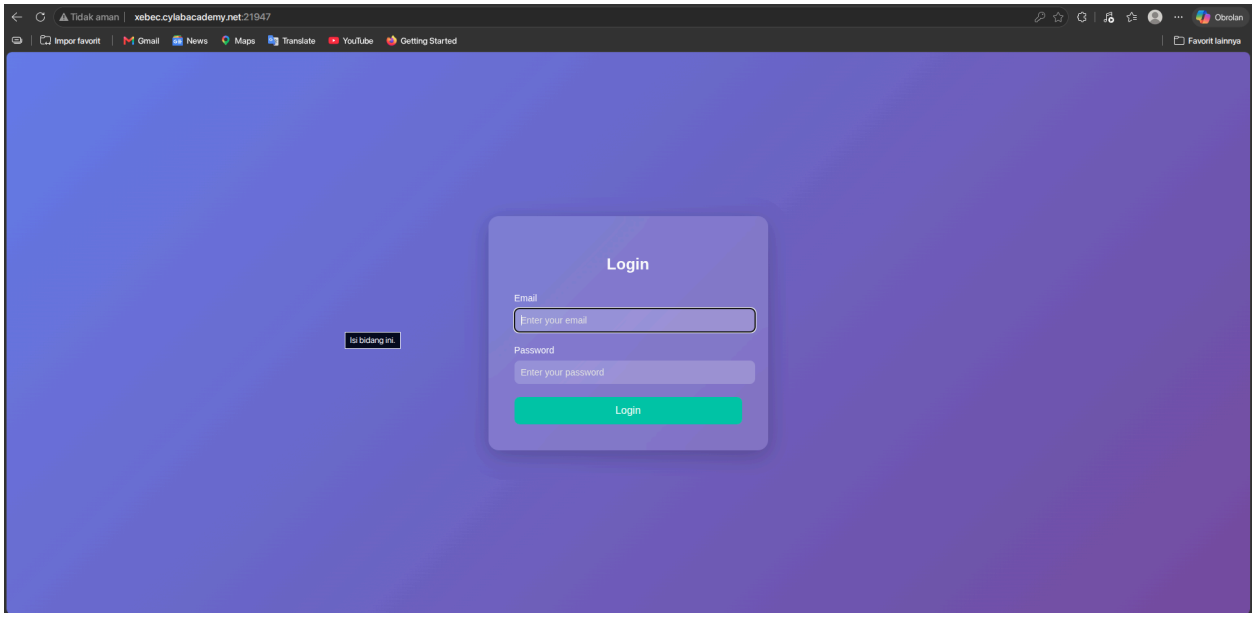
Hashgate

Kategori: Web Exploitation

Deskripsi: You have gotten access to an organisation's portal. Submit your email and password, and it redirects you to your profile. But be careful: just because access to the admin isn't exposed doesn't mean it's secure. Maybe someone forgot that obscurity isn't security... Can you find your way into the admin's profile for this organisation and capture the flag?

Tools yang digunakan: VS Code, Developer Tools

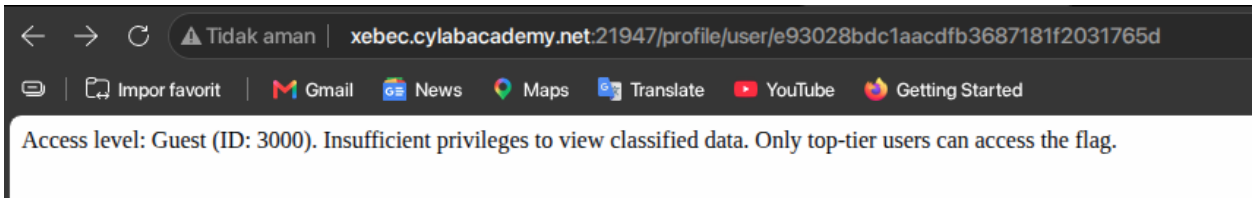
Challenge ini memberikan kita sebuah login page ke sebuah organization, tugas kita adalah mencari profile admin. Berikut adalah tampilan awal pagenya



Coba kita aktifkan developer mode dan eksplor sedikit



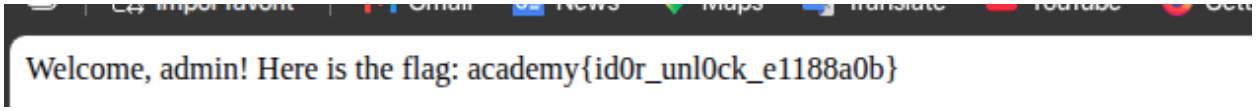
Di kode HTML page, developer meninggalkan kredensial guest untuk site tersebut. Coba kita gunakan untuk login



Kalau kita perhatikan, id kita di URL berupa hash MD5 karena 3000 jadi hash itu e93028bdc1aacdfb3687181f2031765d.
Untuk mencari id admin, kita akan coba id nomor 1-10000 dalam
Untuk mempercepat proses tersebut, saya sudah membuat script dalam python untuk mengecek tiap id dalam bentuk MD5 hash.

```
=== Done. Found 2 valid user(s): ===
  ID  3000 -> http://chatelaine.cylabacademy.net:22223/profile/user/e93028bdc1aacdfb3687181f20
31765d
  ID  3016 -> http://chatelaine.cylabacademy.net:22223/profile/user/53a1320cb5d2f56130ad5222f9
3da374
```

Ditemukan dua user yang valid, yaitu dengan ID 3000 (kita) dan ID 3016 (admin)



Flag: academy{id0r_unl0ck_e1188a0b}

Unminify

Kategori: Web Exploitation

Deskripsi: I don't like scrolling down to read the code of my website, so I've squished it. As a bonus, my pages load faster!

Tools: Developer Tools

Challenge ini memberikan kita akses ke sebuah situs. Berikut adalah tampilannya



Situs itu bilang kalau flag yang kita cari sudah diterima oleh browser. Coba kita gunakan Developer Tools untuk melihat kode HTML dari situs tersebut.

```
<body class="academy{}" style="margin:0">
  <div class="academy{}" style="margin:0;padding:0;background-color:#757575;display:auto;height:4
  0%">
    <a class="academy{}" href="/">
      <img src="academy-wordmark-white.svg" alt="CyLab Academy logo" style="display:inline-block;widt
      h:160px;padding-left:30px">
    </a>
  </div>
  <center>
    <br class="academy{}">
    <br class="academy{}">
    <div class="academy{}" style="padding-top:30px;border-radius:3%;box-shadow:0 5px 10px #0000004d;
    idth:50%;align-self:center">
      <img class="academy{}" src="hero.svg" alt="flag art" style="width:150px;height:150px">
      <div class="academy{}" style="width:85%">
        <h2 class="academy{}">Welcome to my flag distribution website!</h2>
        <div class="academy{}" style="width:70%"> == $0
          <p class="academy{}">If you're reading this, your browser has succesfully received the
          flag.</p>
          <p class="academy{pr3tty_c0d3_c7ba568b}"></p>
          <p class="academy{}">I just deliver flags, I don't know how to read them...</p>
```

Setelah kita lihat lihat sedikit, kita sudah dapat flagnya!

Flag: academy{pr3tty_c0d3_c7ba568b}